Real-Time Embedded Network Security Simulator

By : Shikhar Sahay, Ujjwal Pandit, Ramsha Akbar, & Anwita Padhi

TABLE OF CONTENTS

01 **Abstract**
A concise overview of RT-ENSS, its goals, and key outcomes.

02 **Introduction**
Embedded system vulnerabilities, research gaps, and our simulation solution.

03 **Methodology**
How nodes, scheduler, network, and IDS work together in the simulation.

04 **Simulation Setup**
SystemC environment, node configuration, and network modeling explained.

TABLE OF CONTENTS CONTINUED

05 **Algorithm/Core Logic**
Task scheduling, message handling, and intrusion detection logic in action.

06 **Results and Discussion**
Simulation outputs showing task execution, network activity, and attack detection.

07 **Conclusion**
Key insights on maintaining secure, real-time operation under attacks.

08 **References**
Sources, frameworks, and research cited for RT-ENSS.

Abstract

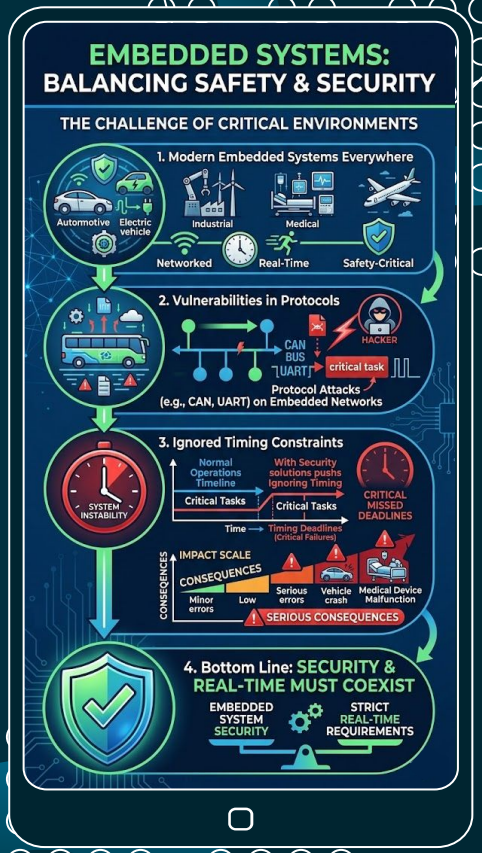
Real-Time Embedded Network Security Simulator (RT-ENSS) is a SystemC-based simulation of a multi-node embedded system executing tasks under a real-time scheduling policy while communicating over a shared embedded network. The system models multiple cyber attacks, including message spoofing, replay attacks, and denial-of-service (DoS), and integrates an intrusion detection mechanism to identify anomalous behavior in real time. By combining task scheduling, network monitoring, and adaptive response strategies, the project demonstrates how embedded systems can maintain security without violating strict timing constraints, with applications in automotive, industrial, and safety-critical domains.

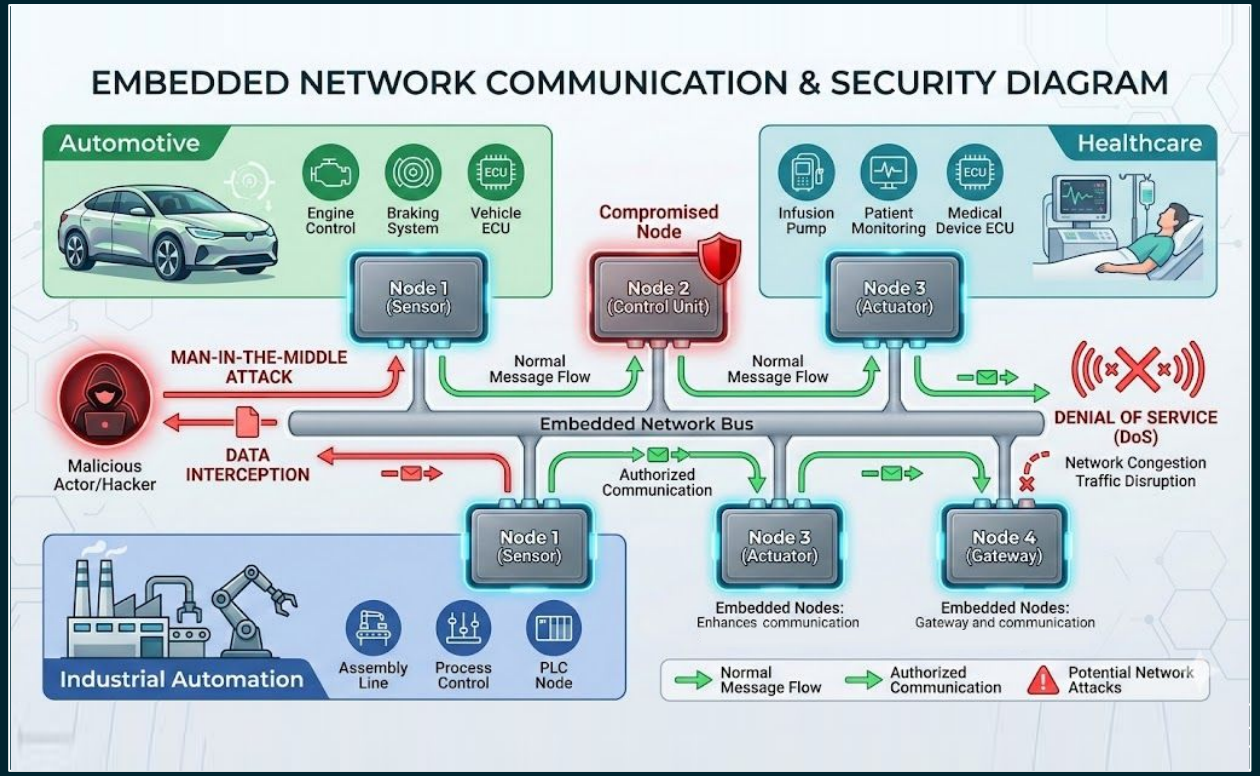
Introduction: Scenario & Background

Modern embedded systems are all around us, from automotive electronics and industrial control systems to medical devices. These systems are networked, real-time, and safety-critical, meaning even small failures or attacks can have serious consequences.

- Cyber attacks on embedded networks (like CAN or UART) can disrupt critical tasks.
- Many current solutions ignore timing constraints, which can cause system instability.

The bottom line being that security in embedded systems must coexist with strict real-time requirements to be effective.





A visual breakdown of message flow and vulnerability in critical embedded networks.

Introduction: Research Gap

Most current embedded system security solutions are hardware-intensive or offline, making them expensive and difficult to prototype for experimentation. Many approaches also ignore real-time scheduling constraints, which is critical in safety-critical applications. Existing studies often focus on cryptographic mechanisms, hardware security modules, or general network intrusion detection systems, but fail to account for real-time behavior or practical evaluation under controlled conditions.

- Hardware IDS solutions are costly and inflexible.
- Offline simulation cannot model real-time deadlines or scheduling behavior.
- Real-time, network-aware security is underexplored.
- Lack of a structured simulation framework that can simulate embedded network attacks and evaluate security mechanisms under timing constraints.

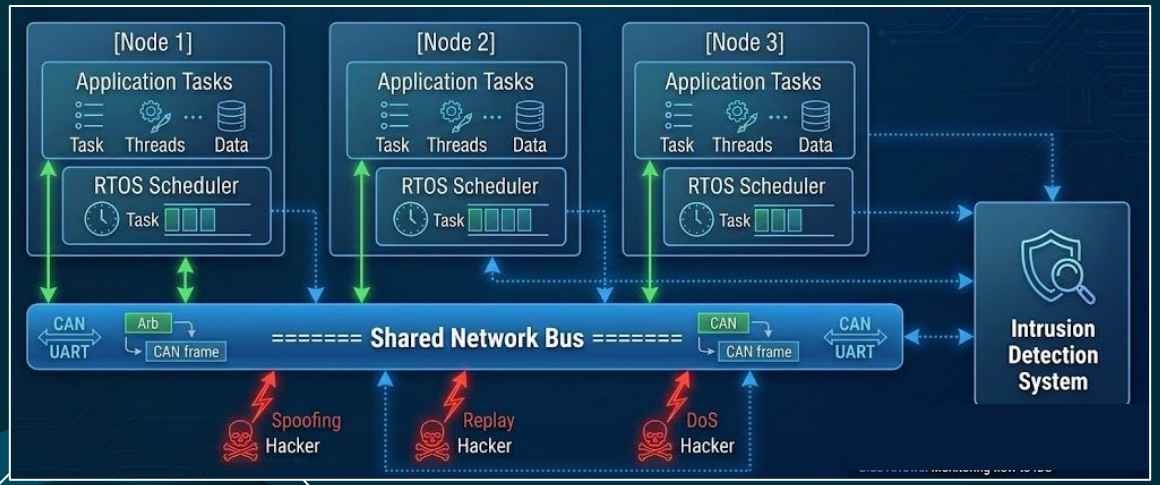
Introduction: Proposed Idea

To bridge the gap between embedded system design and real-time cybersecurity evaluation, we propose RT-ENSS, a SystemC-based simulation framework. It models multi-node embedded systems executing tasks under real-time scheduling, communicates over CAN/UART-like networks, and simulates cyber attacks such as spoofing, replay, and denial-of-service. An intrusion detection system operates within scheduling constraints, allowing evaluation of security mechanisms without violating real-time deadlines. This framework provides a platform to analyze the trade-off between security overhead and task timing performance. Some key features of the proposed implementation include the following:

- Real-time task execution using RMS scheduling
- Multi-node embedded network with structured message communication
- Attack injection module simulating spoofing, replay, and DoS attacks
- Intrusion detection system with real-time anomaly detection and response
- GUI-based dashboard for real-time visualization and analysis

Methodology: How RT-ENSS Works

RT-ENSS simulates a multi-node embedded system where tasks execute under a Rate Monotonic Scheduling (RMS) policy, communicate over an embedded network, and are continuously monitored by an intrusion detection system (IDS). An attack injection module introduces cyber threats during runtime, while each component operates in a closed loop: the scheduler triggers tasks on nodes, nodes exchange messages over the network, the IDS monitors traffic and execution patterns, and system logs feed into visualization and analysis tools. This structure enables the simulation to capture real-time performance, security responses, and task behavior under attack conditions.



Module	Role	Interaction / Function
Node Module	Executes tasks with timers and interrupts.	Sends and receives network messages; responds to scheduler triggers
Scheduler Module	Implements RMS real-time scheduling	Selects and prioritizes tasks each tick to meet timing constraints
Network Module	Simulates CAN/UART-like bus communication	Transfers messages with delay, priority, and contention handling
Attack Injection Module	Generates controlled cyber attacks	Injects spoofed, replayed, or DoS traffic at defined timestamps
IDS Module	Monitors network traffic and task behavior	Detects anomalies based on timing, message patterns, and node activity
Logger & Visualization (Python GUI)	Records and displays system activity	Logs execution timelines, traffic, alerts, and feeds data to the GUI dashboard

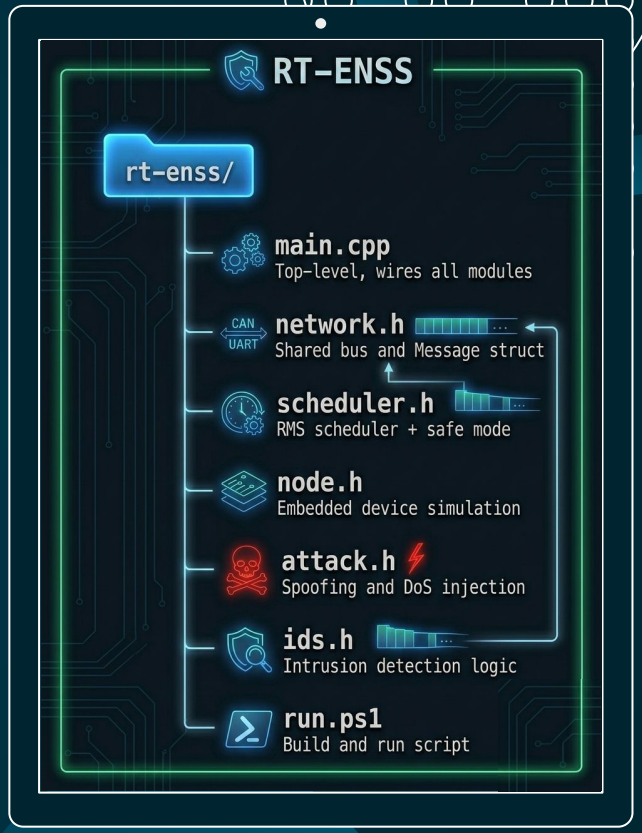
Simulation Setup

The simulation is built on SystemC 3.0.2, compiled using GCC 15.2 via MSYS2, and runs entirely on Windows without any physical hardware or external dependencies. To run the project, you need:

- SystemC 3.0.2 built from source
- GCC 15.2 with C++17 support via MSYS2
- GTKWave for waveform visualization
- PowerShell to execute the included run.ps1 build script
- Python (Tkinter) for GUI-based visualization

The project consists of six header files and a top-level main.cpp for the SystemC simulation, along with a Python-based GUI module for visualization. The run.ps1 script compiles and executes the simulation, generating a CLI event log and a VCD trace for GTKWave.

Four nodes run as concurrent threads, exchanging messages over a shared network bus. Each message includes a sender ID, payload, and timestamp for real-time IDS monitoring. The simulation runs for 300 ms with attack events at predefined timestamps, and the logs are processed by a Python GUI dashboard for visualization and analysis.



Core Logic Behind RT-ENSS

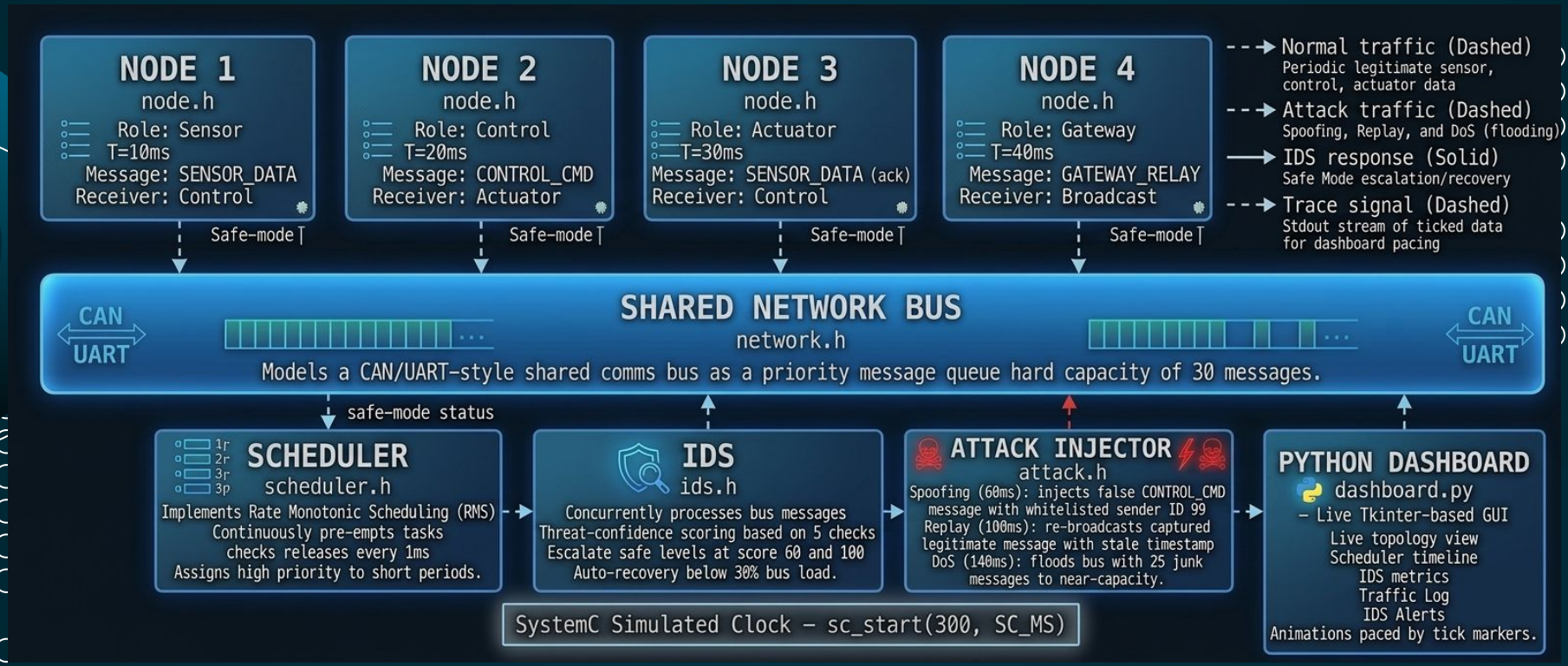
RT-ENSS is designed around a unified real-time execution model where computation, communication, and security operate under the same time constraints. The core logic integrates five tightly coupled modules:

- Scheduler manages real-time task execution using RMS
- Nodes generate and process periodic network messages
- Network simulates a constrained shared communication bus
- Attack module injects controlled cyber disruptions
- IDS monitors traffic and system behavior in real time

All modules execute within a synchronized 300 ms SystemC timeline.

Instead of treating security as an external add-on, RT-ENSS embeds detection directly into the scheduling an communication loop, ensuring intrusion detection operates within bounded latency while preserving task deadlines.

The objective is to demonstrate coordinated behavior across all modules without violating real-time guaran


NODE 1
node.h
Role: Sensor
T=10ms
Message: SENSOR_DATA
Receiver: Control
NODE 2
node.h
Role: Control
T=20ms
Message: CONTROL_CMD
Receiver: Actuator
NODE 3
node.h
Role: Actuator
T=30ms
Message: SENSOR_DATA (ack)
Receiver: Control
NODE 4
node.h
Role: Gateway
T=40ms
Message: GATEWAY_RELAY
Receiver: Broadcast
Normal traffic (Dashed)
Periodic legitimate sensor, control, actuator data
Attack traffic (Dashed)
Spoofing, Replay, and DoS (flooding)
IDS response (Solid)
Safe Mode escalation/recovery
Trace signal (Dashed)
Stdout stream of ticked data for dashboard pacing
Safe-mode
Safe-mode
Safe-mode
Safe-mode
CAN
UART
SHARED NETWORK BUS
network.h
Models a CAN/UART-style shared comms bus as a priority message queue hard capacity of 30 messages.
CAN
UART
safe-mode status
SCHEDULER
scheduler.h
Implements Rate Monotonic Scheduling (RMS)
Continuously pre-empts tasks
checks releases every 1ms
Assigns high priority to short periods.
IDS
ids.h
Concurrently processes bus messages
Threat-confidence scoring based on 5 checks
Escalate safe levels at score 60 and 100
Auto-recovery below 30% bus load.
ATTACK INJECTOR
attack.h
Spoofing (60ms): injects false CONTROL_CMD message with whitelisted sender ID 99
Replay (100ms): re-broadcasts captured legitimate message with stale timestamp
DoS (140ms): floods bus with 25 junk messages to near-capacity.
PYTHON DASHBOARD
dashboard.py
– Live Tkinter-based GUI
Live topology view
Scheduler timeline
IDS metrics
Traffic Log
IDS Alerts
Animations paced by tick markers.
SystemC Simulated Clock – sc_start(300, SC_MS)

Runtime Execution & Detection Logic

The simulation runs over a 300 ms virtual timeline where the scheduler, nodes, network, attack module, and IDS operate concurrently under synchronized SystemC time. The scheduler applies RMS to select node tasks, which generate periodic messages every 10 ms. These messages are transmitted over a constrained shared network, while the IDS continuously monitors network activity and execution patterns in real time.

At 60 ms, the attack module injects a spoofed message with an invalid sender ID. At 100 ms, a replay attack is introduced using previously transmitted messages, and at 140 ms, a DoS attack is simulated through network flooding. The IDS evaluates message validity and traffic behavior in real time, and upon detecting anomalies, signals the system to activate safe mode. The safe mode signal transitions from 0 to 1, while task execution continues under RMS without missing critical deadlines.

All task activity, attack events, and safe mode transitions are logged and processed by the Python-based GUI dashboard, enabling real-time visualization and analysis of system behavior.

Results & Discussions

The simulation was executed over a 300 ms virtual timeline, producing a real-time event log processed and visualized through a Python-based GUI dashboard. Some key observations include:

- Normal operation (0–60 ms): All nodes transmitted messages consistently while the scheduler executed tasks according to RMS priorities without delays.

- Spoofing attack (60 ms): A message from an invalid sender was injected. The IDS detected the anomaly and triggered safe mode shortly after detection.

- Replay attack (100 ms): Previously transmitted messages were replayed, leading to detectable duplicate patterns. The IDS identified the anomaly and reinforced safe mode activation.

- DoS attack (140 ms): The network experienced a surge in traffic simulating a denial-of-service condition. The IDS detected abnormal traffic behavior and triggered system protection mechanisms.

- System stability: Across all attack phases, the scheduler continued executing tasks without missing deadlines, demonstrating preservation of real-time guarantees under adverse conditions.

The GUI visualization confirmed safe mode activation during attack phases and continuous task execution throughout simulation.

Normal Operation (0–60 ms): Nodes exchange messages under RMS scheduling with stable traffic and no anomalies, maintaining a NORMAL safe-state.

Spoofing Attack & Detection (60–80 ms): An invalid sender message is injected, flagged by the IDS, triggering a safe-state transition while task execution continues.

Replay Attack Detection (100–120 ms): Duplicate message patterns are introduced and detected by the IDS without disrupting real-time scheduling.

DoS Attack Response (140–180 ms): A surge in network traffic is identified as a denial-of-service condition, activating protective mechanisms while tasks continue.

Final System State (300 ms): Despite multiple attack phases, the system remains stable with uninterrupted RMS task execution and effective intrusion detection.

Conclusion

RT-ENSS successfully demonstrates that real-time embedded systems can detect and respond to cyber attacks without violating strict timing constraints. The simulation validated three core outcomes:

- The RMS scheduler consistently prioritized and executed tasks on time, even during active attacks and safe mode operation

- The IDS detected spoofing, replay, and DoS attacks in real time, triggering timely system responses

- Task execution continued uninterrupted after safe mode was triggered, proving that security and real-time performance can coexist

The project shows that rule-based intrusion detection is viable within embedded resource constraints, and that SystemC is an effective platform for modeling and validating such systems before physical deployment.

Future extensions could include more advanced IDS techniques (e.g., adaptive or learning-based detection), expand network configurations, and integration with hardware-based embedded systems.

References

[1] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," Journal of the ACM, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[2] G. C. Buttazzo, Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, 3rd ed. New York, NY, USA: Springer, 2011.

[3] IEEE Standard for Standard SystemC Language Reference Manual, IEEE Std 1666-2011, Jan. 2012.

[4] Robert Bosch GmbH, CAN Specification Version 2.0, Sep. 1991.

[5] T. Hoppe, S. Kiltz, and J. Dittmann, "Security threats to automotive CAN networks—Practical examples and selected short-term countermeasures," in Computer Safety, Reliability, and Security (SAFECOMP 2008), LNCS 5219, pp. 235–248, 2008.

[6] K. Koscher et al., "Experimental security analysis of a modern automobile," in 2010 IEEE Symposium on Security and Privacy, pp. 447–462, 2010.

[7] S. Checkoway et al., "Comprehensive experimental analyses of automotive attack surfaces," in Proceedings of the 20th USENIX Security Symposium, 2011.

References Continued

[8] U. E. Larson, D. K. Nilsson, and E. Jonsson, "An approach to specification-based attack detection for in-vehicle networks," in 2008 IEEE Intelligent Vehicles Symposium, pp. 220–225, 2008.

[9] M. Müter, A. Groll, and F. C. Freiling, "A structured approach to anomaly detection for in-vehicle networks," in 2010 Sixth International Conference on Information Assurance and Security, pp. 92–98, 2010.

[10] I. Studnia, E. Alata, V. Nicomette, M. Kaâniche, and Y. Laarouchi, "A language-based intrusion detection approach for automotive embedded networks," International Journal of Embedded Systems, vol. 10, no. 1, pp. 1–12, 2018.

[11] J. Rushby, "Critical system properties: Survey and taxonomy," Reliability Engineering & System Safety, vol. 43, no. 2, pp. 189–219, 1994.

[12] P. Koopman, "Embedded system security," IEEE Computer, vol. 37, no. 7, pp. 95–97, 2004.

[13] E. A. Lee, "Cyber physical systems: Design challenges," in 2008 11th IEEE International Symposium on Object-Oriented Real-Time Distributed Computing (ISORC), pp. 363–369, 2008.

[14] A. Burns and A. J. Wellings, Real-Time Systems and Programming Languages, 4th ed. Addison-Wesley, 2009.

[15] S. Woo, H. J. Jo, and D. H. Lee, "A practical wireless attack on the connected car and security protocol for in-vehicle CAN," IEEE Transactions on Intelligent Transportation Systems, vol. 16, no. 2, pp. 993–1006, 2015.

THANK YOU!

QUESTIONS? — We look forward to your inquiries.